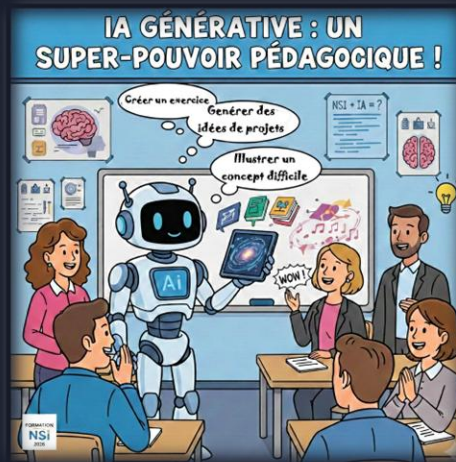


NIVEAU 2 – INTERMÉDIAIRE

L'IA Générative Appliquée à la NSI



Construire des séquences pédagogiques complètes avec l'IA

2 x 3h30 Prérequis : Niveau 1 Ateliers intensifs NSI



Rappel du Niveau 1 et Objectifs du Niveau 2

Ce que vous savez déjà • Ce que vous allez acquérir

Acquis – Niveau 1

- Comprendre ce qu'est un LLM (prédiction de tokens)
- Structure d'un prompt RCTF (Rôle–Contexte–Tâche–Format)
- Générer exercices et corrigés simples en NSI
- Identifier hallucinations et enjeux RGPD

Maîtriser – Niveau 2

- Prompt engineering avancé : few-shot & chain-of-thought
- Créer des séquences pédagogiques NSI complètes
- Générer TP, évaluations, supports de révision
- Analyser et corriger les sorties IA de façon systématique
- Intégrer l'IA dans son flux de travail d'enseignant

Prompt Engineering Avancé

Few-shot prompting et Chain-of-Thought appliqués à la NSI

Few-Shot Prompting

Le few-shot prompting est une technique qui consiste à inclure directement dans votre prompt quelques exemples de ce que vous voulez obtenir, avant de formuler votre véritable demande.

```
"Voici 2 exercices modèles :  
[Ex 1] Contexte: algo BFS  
      Énoncé: ...  
      Corrigé Python: ...  
[Ex 2] Contexte: pile/file  
      Énoncé: ...  
      Corrigé Python: ...  
Génère un exercice identique  
sur les ARBRES BINAIRES."
```

Chain-of-Thought (CoT)

Le Chain-of-Thought (CoT) est une technique utilisée en intelligence artificielle, notamment dans le domaine du traitement automatique du langage naturel (NLP). Elle consiste à décomposer un problème complexe en une série d'étapes de raisonnement intermédiaires, similaires à la façon dont un humain aborderait le problème.

```
"Avant de générer l'exercice,  
raisonne étape par étape :  
1. Identifie les prérequis  
   algorithmes de ce thème  
2. Détermine le niveau de  
   difficulté adapté Terminale  
3. Prévois les erreurs  
   classiques des élèves  
Puis génère l'exercice."
```

Cas Pratique : Séquence Algorithmique Complète

De l'introduction au TP évalué – Graphes et plus court chemin

1

Cours magistral

1h

"Génère un cours de 2 pages sur Dijkstra : définition, pseudo-code annoté, exemple graphe 5 nœuds, complexité $O((V+E)\log V)$ expliquée"

2

Exercices guidés

45min

"Crée 3 exercices progressifs (facile/moyen/difficile) sur Dijkstra avec graphes ASCII, questions guidées et corrigés Python commentés"

3

TP noté

1h30

"Génère un TP évalué : carte d'une ville (9 nœuds), fichier CSV fourni, script Python incomplet à compléter, barème détaillé 20 pts"

4

Auto-évaluation

20min

"Produis 5 questions flash de consolidation sur Dijkstra vs Bellman-Ford, format vrai/faux + justification courte"

Atelier : Générer un TP Complet – Structures de Données

Piles, files, listes chaînées – Prompt → Ressource prête à l'emploi

Prompt utilisé

"Tu es un prof de NSI Terminale. Crée un TP complet sur les piles (stack) en Python :
1. Implémentation avec liste Python (push/pop/peek/isEmpty) – code commenté
2. Application : vérification de parenthèses équilibrées dans une expression – énoncé + corrigé
3. Exercice bonus : simuler une pile avec deux files – énoncé guidé en 4 étapes
Format : fichier Python annoté + PDF énoncé séparé."

Classe Stack

```
class Stack:
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item)
    def pop(self):
        return self.items.pop()
    # peek, isEmpty...
```

Application métier

```
def parentheses_ok(expr):
    pile = Stack()
    for c in expr:
        if c in '([{':
            pile.push(c)
        elif c in ')]}' and pile.isEmpty():
            return False
    return pile.isEmpty()
```

Bonus guidé

Étape 1 : Créer file1 et file2
Étape 2 : push → enfiler dans file1
Étape 3 : pop → transvaser
Étape 4 : peek sans dépiler
→ Complexité amortie $O(n)$?

Cybersécurité NSI : Générer des Scénarios Réalistes

Cryptographie, protocoles, attaques — ressources prêtes à enseigner

Cryptographie RSA

Prompt :

"Crée un exercice complet sur RSA :
génère $p=17$, $q=11$, calcule n , $\phi(n)$,
 e , d . Guide l'élève pas à pas.
Corrigé Python avec bibliothèque
sympy. Niveau Terminale."

Sortie générée :

Exercice dirigé 5 étapes
Code Python sympy inclus
Barème 12 pts

Protocoles TCP/IP

Prompt :

"Génère un scénario d'analyse de
trame Wireshark simulée : HTTP sur
TCP, 3-way handshake, fragmentation
IP. Questions progressives avec
captures ASCII annotées."

Sortie générée :

Trame ASCII annotée
8 questions de 1→3 pts
Corrigé complet

Attaques & Défenses

Prompt :

"Crée un TP d'investigation : Log
d'attaque SQL injection fourni
(CSV), l'élève identifie le vecteur,
propose la correction SQL paramétré.
Grille d'évaluation 20 pts."

Sortie générée :

Fichier CSV de logs
Guide investigation
Correction préparée

Bases de Données & SQL — Ressources Pédagogiques

Du schéma relationnel au TP d'interrogation avancée

Schéma relationnel généré par IA

Prompt : "Génère un schéma SQL pour une bibliothèque scolaire : tables Livre, Élève, Emprunt, Auteur. Clés primaires/étrangères. 3 requêtes progressives + corrigé."

—— Résultat ——

```
CREATE TABLE Livre (  
  isbn CHAR(13) PRIMARY KEY,  
  titre VARCHAR(200),  
  id_auteur INT REFERENCES Auteur  
);  
CREATE TABLE Emprunt (  
  id_emprunt SERIAL PRIMARY KEY,  
  isbn CHAR(13) REFERENCES Livre,  
  id_eleve INT REFERENCES Eleve,  
  date_retour DATE  
);
```

3 niveaux de requêtes SQL

Niveau 1

```
SELECT titre FROM Livre  
WHERE isbn IN (  
  SELECT isbn FROM Emprunt  
  WHERE id_eleve = 42);
```

Niveau 2

```
SELECT nom, COUNT(*) AS emprunts  
FROM Eleve  
JOIN Emprunt USING(id_eleve)  
GROUP BY nom  
HAVING COUNT(*) > 3;
```

Niveau 3

```
SELECT titre, AVG(duree) AS duree_moy  
FROM Livre JOIN Emprunt ON ...  
WHERE date_retour IS NOT NULL  
GROUP BY titre ORDER BY 2 DESC;
```

Réseaux & Architecture — Ressources Pédagogiques NSI

Modèle OSI, protocoles, simulation de réseau avec l'IA

Modèle OSI / TCP-IP

Prompt : "Crée un tableau comparatif OSI vs TCP/IP avec pour chaque couche : rôle, protocoles NSI, exemple concret lycée.
Format : tableau Markdown exportable."

- Tableau 7 couches prêt à projeter
- Exercice d'identification de couche
- QCM 5 questions avec corrigé

Adressage IP et Masques

Prompt : "Génère 5 exercices progressifs sur le découpage réseau CIDR : /24 → /26, calcul de plages, sous-réseaux. Inclure représentation binaire et tableau récapitulatif."

- 5 exercices avec fichiers .csv
- Script Python de vérification
- Corrigé étape par étape

Routing et Protocoles

Prompt : "Simule un réseau de 6 routeurs en ASCII, génère la table de routage RIP initiale, puis après convergence. Questions sur la détection de boucle. Corrigé complet."

- Schéma réseau ASCII annoté
- Tables de routage évolutives
- TP de dépannage réseau

Complexité Algorithmique & Récursivité

Générer des progressions pédagogiques rigoureuses avec l'IA

Tableau de complexités généré

Algorithme	Temps	Espace
Recherche linéaire	$O(n)$	$O(1)$
Recherche dichotomique	$O(\log n)$	$O(1)$
Tri par sélection	$O(n^2)$	$O(1)$
Tri fusion	$O(n \log n)$	$O(n)$
Dijkstra (tas)	$O((V+E)\log V)$	$O(V)$
BFS / DFS	$O(V+E)$	$O(V)$

Récursivité — Prompt CoT

Prompt : "Raisonne étape par étape.
1. Cas de base de factorielle ?
2. Pile d'appels pour $n=4$?
3. Génère exercice : Fibonacci récursif + mémorisation
4. Visualise l'arbre d'appels en ASCII pour $n=5$ "

Sortie générée :

```
fib(4)
├─ fib(3)
│   └─ fib(2) → 1
│       └─ fib(1) → 1 → 2
└─ fib(2) → 1 → 3
```

Générer des Évaluations & Différencier les Parcours

Adapter le niveau, créer des barèmes, personnaliser les parcours élèves

Aide (A)

Prompt : "Génère le même exercice sur les arbres binaires mais avec : squelette de code Python à compléter, indices progressifs, explications inline."

- Code préstructuré
- Commentaires guidants
- Tests unitaires fournis

Standard (B)

Prompt : "Exercice standard arbres binaires : insérer, parcourir en largeur, compter feuilles. Énoncé seul, corrigé séparé."

- 3 questions de difficulté croissante
- Barème 20 pts détaillé
- Corrigé enseignant

Approfondis. (C)

Prompt : "Version avancée : arbre AVL, équilibrage automatique, démonstration de la complexité $O(\log n)$. Introduire la notion de facteur d'équilibre."

- Exercice ouvert
- Piste de recherche
- Lien vers ressources+

Atelier Pratique – Construire une Séquence NSI de A à Z

Travail en binômes • 3h30 de production réelle

1. Choix du thème & plan

30 min

- Choisir un chapitre du programme NSI Terminale
- Générer un plan de séquence avec l'IA (4-6 séances)
- Identifier les prérequis algorithmes / structures

2. Production des ressources

1h30

- Générer : cours, 3 exercices, 1 TP noté, 1 évaluation
- Utiliser few-shot + CoT pour chaque ressource
- Itérer : détecter erreurs, corriger les prompts

3. Vérification critique

45 min

- Tester le code Python généré dans un notebook
- Vérifier la rigueur mathématique (complexité, preuves)
- Annoter les erreurs et reformuler les prompts

4. Partage & retour collectif

45 min

- Présenter 1 ressource au groupe (5 min)
- Débat : quelle part du travail reste à l'enseignant ?
- Construire un prompt template réutilisable

Vérifier & Corriger les Sorties IA Systématiquement

Un processus en 4 étapes pour valider chaque ressource générée

1 Exécuter le code



Toujours tester le code Python dans un notebook ou Thonny. L'IA fait des erreurs d'indentation, d'import, de logique. 1 bug sur 3 exercices en moyenne.

2 Vérifier la rigueur



Contrôler : complexité annoncée ($O(n)$ vs $O(n^2)$), cas limites (liste vide, $n=0$), et cohérence avec le programme officiel NSI Terminale.

3 Tester avec les élèves



Une ressource validée techniquement peut être pédagogiquement mal calibrée. Faire relire par un collègue ou tester sur un groupe avant diffusion.

4 Améliorer le prompt



Conserver les prompts qui ont bien fonctionné dans un fichier partagé. Documenter les échecs. Construire une bibliothèque de prompts NSI collective.

Intégrer l'IA dans son Quotidien d'Enseignant NSI

Quand utiliser l'IA, quand ne pas l'utiliser

IA recommandée pour...

- ✓ Générer des variantes d'exercices (même concept, données différentes)
- ✓ Créer rapidement un corrigé-type à partir d'un énoncé existant
- ✓ Adapter le niveau d'un exercice (simplifié / enrichi)
- ✓ Rédiger le plan d'une séquence ou d'une progression
- ✓ Produire des questions flash de révision (flashcards)
- ✓ Générer des exemples concrets pour illustrer un concept abstrait

Prudence / Ne pas déléguer à l'IA

- ✗ La vérification finale (toujours relire et tester le code généré)
- ✗ Le choix pédagogique : quelle notion enseigner, dans quel ordre
- ✗ L'évaluation finale officielle (créez-la vous-même, IA = aide brouillon)
- ✗ Les données personnelles d'élèves (jamais dans un prompt IA externe)
- ✗ Les preuves mathématiques formelles (vérifiez ligne par ligne)
- ✗ Le jugement sur la compréhension d'un élève (l'IA ne connaît pas votre classe)

Ce que vous maîtrisez maintenant

Niveau 2 – Intermédiaire NSI

- ✓ Prompt engineering avancé (few-shot, chain-of-thought)
- ✓ Séquences pédagogiques complètes : cours → TP → évaluation
- ✓ Ressources NSI : algo, BDD, réseaux, cybersécurité, complexité
- ✓ Différenciation automatisée des parcours élèves
- ✓ Processus de vérification systématique des sorties IA

Prochain niveau – Avancé

- › Fonctionnement des transformers (attention, tokenisation)
- › Projets élèves autonomes : coder avec l'IA
- › Fine-tuning, RAG, agents IA — panorama et usages NSI
- › IA Act et cadre légal européen

